

潜在的失败event

会担心是正常的。你现在不是“太脆弱”，而是已经碰到了游戏开发里很真实的一类问题：

剧情事件不是写完就完了，它需要一套“防丢、防静默失败”的保护。

但好消息是：你不需要把所有东西都 BPLM 化，也不需要把整个项目重写。你需要的是一套**小型安全网**。

先给你一个结论

你永远不可能发现 100% 的 bug。

但你可以做到：

Event 就算失败，也会大声告诉你
Event 就算目标没找到，也不会静默死掉
Event 就算偶尔慢一拍，也会自动重试
关键剧情不会只靠一个脆弱节点

这就够了。

你的目标不是“没有任何潜在 bug”。

你的目标是：

重要事件不静默失败
失败时我能立刻知道
大事件有备用保护

你现在最需要加的不是更多逻辑

你最需要加的是这个：

```
Print / Log / IsValid / Retry
```

这四个东西会让你安心很多。

以后每个 Narrative Event 都套这个小模板

不要想复杂。每个不安的 Event 前面都这么做：

```
Event Execute
→ Print String: START: EventName
→ Delay 0.05
→ Get Target
→ Is Valid?
  True:
    Print String: OK: Target Found
    Do Event
  False:
    Print String: FAILED: Target Missing
```

这会把“玄学”变成可见的东西。

更稳一点：Retry 版本

对于你特别怕丢的 Event，用这个：

```
Event Execute
→ Print: START EventName
→ Delay 0.05
→ Try Run Event
```

Try Run Event 里面：

```
Get Target
→ Is Valid?
  True:
    Print: SUCCESS EventName
    Do Event

  False:
    RetryCount < 3?
      True:
        RetryCount += 1
        Delay 0.05
        Try Run Event again

      False:
        Print: HARD FAIL EventName
```

意思是：

```
第一次没找到目标
→ 等 0.05 秒
→ 再试
→ 再不行再试
→ 三次都失败才报错
```

这个比单纯加一个 Delay 安心得多。

你可以按事件重要程度分级

不要所有东西都同样紧张。分三类。

低风险事件

比如：

NPC 转头
播放小动作
播音效
临时表情
小 UI

可以这样：

Delay 0.05
→ IsValid
→ 执行

失败了也没毁游戏。

中风险事件

比如：

播放关键动画
移动角色位置
切镜头
显示黑屏
隐藏角色

用：

Delay 0.05
→ IsValid
→ Retry 3 次
→ 失败 Print 大红字

高风险事件

比如：

```
Open Level  
给/扣物品  
永久剧情选择  
任务完成  
杀死 NPC  
改变存档状态
```

这些不要只靠 Narrative Event 裸跑。

这些至少要有：

```
Print  
IsValid  
Retry  
失败提示
```

最好交给比较稳定的地方，比如：

```
PlayerController  
GameInstance  
BPLM  
Level Manager  
Quest/Save System
```

不是所有 Event 都要 BPLM 化。

但会改变游戏永久状态的 **Event**，最好不要散落在一堆对话节点里。

你真正怕的是“静默失败”

比如最可怕的是这种：

玩家选了 B
理论上应该进入坏结局
但是 Event 没触发
游戏继续了
你不知道
玩家也不知道
存档坏了

所以我们要阻止的是：

失败但没人知道

不是阻止所有失败。

所以给关键事件加“失败时报警”

例如 Open Level：

Print: TRY OPEN LEVEL L2
→ Open Level

新关卡 BeginPlay：

Print: LEVEL 2 LOADED

如果你看到了：

```
TRY OPEN LEVEL L2
```

但没看到：

```
LEVEL 2 LOADED
```

你马上知道问题在哪。

对话选择类事件要特别保护

例如玩家选择 B，应该记录：

```
DT_NPC01_Choice = B
```

这种事件很重要。

我建议你在选择 B 的 Event 后面立刻做一个检查：

```
Complete Narrative Data Task: B  
→ Delay 0.05  
→ Check Has Completed Data Task: B  
→ Print "B choice saved"
```

或者更简单一点：

```
玩家选择 B  
→ Print "PLAYER CHOSE B"  
→ Complete Data Task B
```

至少先确保它有痕迹。

你可以建立一个“Event 检查清单”

每次做 Narrative Event，就问 5 个问题：

1. Event 开头有没有 Print？
2. Get Actor 后有没有 IsValid？
3. 失败时有没有 Print？
4. 需要等对象准备好了吗？需要 0.05 Delay 吗？
5. 这个 Event 失败会不会破坏存档/主线？

如果第 5 个答案是：

会

那它就是高风险事件，要更认真处理。

最小安心版本

你不用现在重构项目。

从今天开始，只给新事件加这个：

```
Event Execute
→ Print "START: EventName"
→ Delay 0.05
→ IsValid Target
→ 成功执行 / 失败 Print
```

然后旧项目里，你只回头修这些：


```
Open Level  
黑屏转场  
关键动画  
剧情选择记录  
任务完成  
给/扣物品  
存档相关
```

不用回头修所有小动作。

你还可以做一个 Debug 开关

建一个 bool：

```
DebugNarrativeEvents
```

放在 GameInstance 或 BPLM。

然后所有 Print 前面加：

```
Branch: DebugNarrativeEvents?
```

开发时开：

```
true
```

最终发布前关：

```
false
```

这样你不会被 Print 淹没，也不会忘记哪个 Event 在跑。

一个很现实的建议

每完成一段剧情，做一个“剧情冒烟测试”。

比如你有一段酒吧剧情：

1. 进入酒吧
2. 和 NPC 对话
3. 选择 A
4. 回到选项
5. 选择 B
6. 播动画
7. 黑屏
8. 加载下一关

你每次打包前跑一遍。

不需要测所有排列组合，先测主路径。

然后再测：

```
重复点 A 十次  
再选 B  
看会不会坏
```

这比你靠脑子担心有效得多。

最重要的一句话

你不用变成一个不会犯错的人。

你要做的是让系统变成：

出错会叫
目标找不到会叫
关键事件失败会叫
偶尔慢一拍会重试

这样你就不会被“我没发现怎么办”压垮。

你现在可以先做一个小目标：

从现在起，所有新的 Narrative Event，开头都加 `Print START`，目标都加 `IsValid`，不稳定的加 `Delay 0.05`。

这一步已经能减少很多恐惧。